

To add custom headers to the `next-intl` middleware that your Server Components can later read, you must modify the request before or after the `next-intl` middleware processes it and ensure the headers are passed through a Next.js Rewrite. [🔗](#)

1. Configure Middleware [🔗](#)

In your `middleware.ts`, wrap the `next-intl` middleware to inject custom headers into the request. You must set these headers on a new `Headers` object and pass them back via `NextResponse.next` or a rewrite to make them accessible to Server Components. [🌐 next-intl +1](#)

typescript

```
import createMiddleware from 'next-intl/middleware';
import { NextRequest, NextResponse } from 'next/server';

const handleI18nRouting = createMiddleware({
  locales: ['en', 'de'],
  defaultLocale: 'en'
});

export default function middleware(request: NextRequest) {
  // Step 1: Run the i18n middleware
  const response = handleI18nRouting(request);

  // Step 2: Add custom request headers
  // Note: Request headers must be set on the request object passed to the next step
  const requestHeaders = new Headers(request.headers);
  requestHeaders.set('x-custom-header', 'your-value');

  // Step 3: Return a response that includes these new request headers
  // For Server Components to see them, use NextResponse.next with the new headers
  return NextResponse.next({
    request: {
      headers: requestHeaders,
    },
  });
}
```

2. Read Headers in Server Components

Once set in the middleware, you can retrieve these headers in any Server Component using the [Next.js headers function](#). [🔗](#)

tsx

```
import { headers } from 'next/headers';

export default async function Page() {
  const headerList = await headers();
  const customHeader = headerList.get('x-custom-header');

  return <div>Header value: {customHeader}</div>;
}
```

Key Considerations

- **Execution Order:** If you need the `next-intl` middleware to decide the locale based on a custom header, set the header **before** calling `handleI18nRouting(request)`.
- **Response vs. Request Headers:** Setting `response.headers.set()` makes headers available to the browser (client), but it does **not** make them available to your Server Components. Use the `NextResponse.next({ request: { headers } })` pattern shown above to pass data to the server.
- **Dynamic Rendering:** Using the `headers()` function in a Server Component will opt that page into Dynamic Rendering. [GitHub +4](#)